

AdSA-ALMD User Guide

From ABS Census data to a labour market delineation and back

adsalmd package – internal tooling reference (dev/)

2026-09-18

Contents

0. Prerequisites	2
1. Prepare your Census extract	3
1.1 Filter to employed, working-age, non-imputed records	3
1.2 Resolve place-of-work from DZN to SA2	4
1.3 Fix the canonical SA2 order for this run	4
1.4 Build the OD matrix W	4
1.5 Remove BGUs with no commuting ties at all	4
1.6 (Optional but recommended) look at self-containment	5
1.7 Sample-data walkthrough (expected output)	6
2. Build the adjacency matrix	6
3. Validate before you run anything expensive	7
4. Run the pipeline	7
4.0 First, sanity-check the pipeline itself on prepared test data	7
4.1 Run it on your real data	8
What you get back	8
5. Convert the result back into ABS correspondence format	9
5.1 Reattach the BGUs you set aside in step 1.5	9
5.2 Build the correspondence table	9
5.3 (Optional) attach per-LMA quality metrics	10
5.4 Export	10
5.5 (Optional) attach cluster IDs to the shapefile, for mapping	10
6. Full worked example (one script)	11
7. Troubleshooting	12
8. Where each piece lives	13

This is an end-to-end walkthrough for someone who has ABS Census microdata (PLIDA / DataLab) and wants to:

1. turn it into the `W / adj` inputs the `adsalmd` package needs,
2. run the delineation pipeline (`run_adsa_pipeline()`), and
3. convert the result back into an ABS-style correspondence table — the same row-per-region, code/name/ratio layout as an official ABS geography correspondence file — so the output can be read, joined, and reported on the way ABS geography products normally are.

Everything in this guide, and every function it calls from `dev/`, is internal tooling — not part of the installable `adsalmd` package. The package itself (`R/`) only knows about `W`, `adj`, and numeric thresholds; it has no ABS-specific code. `dev/` bridges the gap for this specific data source. See `dev/README.md` for the shorter reference version of steps 1–2 below.

0. Prerequisites

Working directory

Set your R working directory to the root of this repository before running anything below — every path in this guide ("`dev/...`", "`examples/...`", "`my_census_extract.csv`", etc.) is written relative to that root, not to wherever R happened to start:

```
setwd("C:/path/to/your/checkout/of/AdSA-ALMD")
# ^ placeholder -- point this at wherever you cloned/downloaded this repo
getwd() # sanity check: should print the folder containing dev/, R/, examples/
```

If you opened this project via the `AdSA-ALMD.Rproj` file in RStudio, your working directory is already set correctly and you can skip this step.

R packages - Base R covers everything except the adjacency step. - `sf` and `spdep`, only for building `adj` from a shapefile.

```
install.packages(c("sf", "spdep"))
```

Data you need in hand

#	What	Where it comes from
1	Person-level Census microdata	PLIDA / DataLab extract, or a CSV with the same columns
2	DZN → SA2 concordance	ABS-published <code>DZN_SA2_<year>_AUST.csv</code>
3	SA2 boundary shapefile	ABS ASGS digital boundary files (<code>SA2_<year>_AUST_GDA<datum>.shp</code>)

None of these three are included in this repo — #2 and #3 are ABS-published reference files you download yourself (search the ABS website for “ASGS digital boundary files” and the DZN/SA2 correspondence under Census geography products), and #1 is your own extract. The paths below ("`DZN_SA2_2016_AUST.csv`" etc.) are placeholders — swap in wherever you actually saved each file on your machine.

All paths in this guide are relative to the repository root — the directory you’d `source("dev/...")` from below, *not* the `dev/` folder that `USER_GUIDE.md` itself lives in.

Source the helper functions (adjust the path to wherever your checkout of this repo lives):

```
source("dev/abs_census_to_od.R")
source("dev/abs_adjacency_from_shapefile.R")
source("dev/validate_lma_inputs.R")
source("dev/lma_result_to_abs_correspondence.R")

library(adsalmd)  # the installed package, for run_adsa_pipeline() etc.
```

1. Prepare your Census extract

If your data arrives as plain CSV rather than a DataLab SAS extract, it still needs these exact ABS Census Dictionary column names (or you pass the matching names as arguments — see step-by-step below):

Header	Meaning	Notes
ABSPID	Person identifier	carried through, not filtered on
AGEP	Age	used for the working-age filter
LFSP	Labour force status	1/2/3 = employed full-time / part-time / away from work
PURP	Place of Usual Residence	already SA2-coded
POWP	Place of Work	DZN -coded, needs conversion (step 1.2)
IFPOWP	Imputation flag for POWP	0 = observed, 1 = imputed

Don't have an extract to hand yet? `dev/sample_data/` ships a small synthetic one (1,000 fake person records, 10 Brisbane SA2s) with these exact column names and coding scheme, plus a matching DZN → SA2 lookup, so you can run this entire section — and check the helper functions actually work — before you've pulled a real DataLab extract:

```
census_raw      <- read.csv("dev/sample_data/census_extract_sample.csv", colClasses = "character")
dzn_sa2_lookup  <- read.csv("dev/sample_data/DZN_SA2_2016_AUST_sample.csv", colClasses = "character")
```

Everything from here through 1.6 runs unchanged on this sample data (step 1.5 is a no-op for it — see the note there); see 1.7 below for the full sample walkthrough with expected output. When you're ready to switch to your own data, it's a one-line swap — same column names throughout:

```
census_raw <- read.csv("my_census_extract.csv", colClasses = "character")
```

Read AGEP as numeric if it came in as character:

```
census_raw$AGEP <- as.integer(census_raw$AGEP)
```

1.1 Filter to employed, working-age, non-imputed records

```
census_filtered <- filter_census_employed(
  census_raw,
  age_col = "AGEP", age_min = 15, age_max = 64,
  lfsp_col = "LFSP", lfsp_employed = c("1", "2", "3"),
  ifpowp_col = "IFPOWP", require_non_imputed = TRUE
)
```

If your extract already excludes imputed records, or uses different age bounds, adjust the arguments — nothing here is hardcoded.

1.2 Resolve place-of-work from DZN to SA2

```
dzn_sa2_lookup <- read.csv("DZN_SA2_2016_AUST.csv", colClasses = "character")
# ^ placeholder -- point this at wherever you saved the ABS-published concordance
# expects columns DZN_CODE_2016, SA2_MAINCODE_2016 by default -- rename or
# pass dzn_lookup_col / sa2_lookup_col if your file uses different headers

census_sa2 <- convert_powp_to_sa2(census_filtered, dzn_sa2_lookup)
```

(If you loaded `dzn_sa2_lookup` from `dev/sample_data/` above, skip the `read.csv()` line here — it's already in your environment — and go straight to `convert_powp_to_sa2()`.)

`census_sa2` now has `origin_SA2` (= PURP) and `destination_SA2` (from POWP via the DZN lookup). Rows whose POWP didn't match any DZN in the lookup are dropped by default (`drop_unmatched = TRUE`).

1.3 Fix the canonical SA2 order for this run

`W` and `adj` must agree, row for row, on which SA2 is unit *i*. Decide that order once, here, and reuse it everywhere below:

```
sa2_codes <- sort(unique(c(census_sa2$origin_SA2, census_sa2$destination_SA2)))
```

(Or supply your own study-area list of SA2 codes if you want to include SA2s with zero recorded flow, or exclude some.)

1.4 Build the OD matrix `W`

```
W <- build_od_matrix(census_sa2, sa2_codes)
# W[i, j] = number of employed residents of sa2_codes[i] working in sa2_codes[j]
```

1.5 Remove BGUs with no commuting ties at all

This only matters if `sa2_codes` (step 1.3) covers more than what shows up in `census_sa2` — i.e. you took the note above and supplied your own full study-area list of SA2 codes (the right thing to do for a real ABS-style correspondence product, since the final table should cover every SA2 in scope, not just the ones with recorded commuters). If you built `sa2_codes` the default way, purely from `census_sa2`'s own values, skip this step — every SA2 in it already has at least one recorded resident or worker by construction.

A full study-area list will usually include a few SA2s where literally nobody lives **and** nobody works — a national park, an industrial or port zone, an uninhabited island. In `W` these show up as an all-zero row *and* an all-zero column: no in-commuting, no out-commuting, no tie to the network at all. That's a different problem from low self-containment (§1.6 below) — a poorly self-contained SA2 still has commuters flowing in and out, which is exactly what SARA's merging step is for. A BGU with **no** flow at all gives the pipeline nothing to decide *which* market it belongs to, so it needs to come out before you build `adj` or run anything:

```
row_W <- rowSums(W)
col_W <- colSums(W)
empty_bgu <- sa2_codes[row_W == 0 & col_W == 0]

if (length(empty_bgu) > 0) {
  message(length(empty_bgu), " SA2(s) have no residents and no jobs recorded -- ",
    "setting aside: ", paste(empty_bgu, collapse = ", "))
}

sa2_codes <- setdiff(sa2_codes, empty_bgu)
W <- W[sa2_codes, sa2_codes]
```

Keep `empty_bgu` (even if it's `character(0)`) — you'll need it in step 5.1 to fold these SA2s back into the final correspondence table by geographic proximity, since they have no commuting flow to place them by. Build `adj` (§2 below) from this same trimmed `sa2_codes`, not the original one.

1.6 (Optional but recommended) look at self-containment

This is the same self-containment figure the pipeline's `SC_min` threshold is checked against, so looking at it now tells you what thresholds are even achievable for your data before you spend a run finding out:

```
sc_table <- compute_self_containment(W)
summary(sc_table$SC)
```

Getting a data-driven `Pop_min`, not a guess

`Pop_min` is a floor on final *LMA* population, but `sc_table` only has per-SA2 numbers — so don't read `Pop_min` straight off it. What it does give you is a defensible starting floor: population and self-containment are correlated (small SA2s leak commuters to bigger neighbours), so the smallest population among the SA2s that *already* clear your chosen `SC_min` on their own tells you the smallest population at which adequate self-containment is empirically achievable in this data:

```
SC_min <- 0.55 # whatever threshold you settled on above

self_contained <- sc_table[!is.na(sc_table$SC) & sc_table$SC >= SC_min, ]
Pop_min <- if (nrow(self_contained) > 0) min(self_contained$W_iG) else NA
Pop_min
```

Because SARA only ever *merges* SA2s to fix a market that's too small or not self-contained enough (see §4), every LMA in the final result will have a population at least this large — this is a genuine floor, not a target. If `self_contained` comes back empty (common on small samples, e.g. the bundled `dev/sample_data/`, where `SC` never gets near 0.55 — see 1.7 below), no SA2 clears the bar on its own; fall back to a policy-based minimum instead (a stated ABS/labour-market adequacy convention, or the value used in a prior study of the same region) until you have real data to check it against. Once you have an actual `res$best_sol`, `suggest_thresholds()` (§4.0) gives you a second, independent read on `Pop_min` from the realised partition — the two won't match exactly, but they should be in the same ballpark; a large gap is worth investigating before you trust the run.

1.7 Sample-data walkthrough (expected output)

Running 1.1–1.6 above against `dev/sample_data/` end to end (the sample data's `sa2_codes` is derived purely from `census_sa2`, so step 1.5 has nothing to remove here — it's not shown below):

```
source("dev/abs_census_to_od.R")

census_raw      <- read.csv("dev/sample_data/census_extract_sample.csv", colClasses = "character")
census_raw$AGEP <- as.integer(census_raw$AGEP)
dzn_sa2_lookup  <- read.csv("dev/sample_data/DZN_SA2_2016_AUST_sample.csv", colClasses = "character")

census_filtered <- filter_census_employed(census_raw)
census_sa2      <- convert_powp_to_sa2(census_filtered, dzn_sa2_lookup)
sa2_codes       <- sort(unique(c(census_sa2$origin_SA2, census_sa2$destination_SA2)))

W              <- build_od_matrix(census_sa2, sa2_codes)
sc_table <- compute_self_containment(W)
```

gives you:

```
> nrow(census_raw)
[1] 1000
> nrow(census_filtered)      # after the employed / working-age / non-imputed filter
[1] 400
> dim(W)
[1] 10 10
> sum(W)                     # equals nrow(census_sa2) -- every filtered record landed somewhere
[1] 400
> summary(sc_table$SC)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.02381 0.03237 0.05583 0.07002 0.11076 0.12963
```

Self-containment is low across the board here because the sample data assigns PURP/POWP uniformly at random — real commuting data is far more spatially clustered. Don't read anything into the numbers themselves; the point is that the pipeline runs and `W` comes out the right shape. `dev/sample_data/generate_sample_data.R` regenerates both CSVs if you want to tweak the scenario (more SA2s, a different sample size, etc).

2. Build the adjacency matrix

Using an ABS SA2 boundary shapefile and queen contiguity (shared edge or vertex — the ABS/LMA-delineation convention):

```
adj <- build_adjacency_from_shapefile(
  shapefile_path = "SA2_2016_AUST_GDA2020.shp",
  # ^ placeholder -- point this at wherever you saved the ABS ASGS boundary
  # files (.shp needs its .dbf/.shx/.prj siblings in the same folder)
  sa2_codes      = sa2_codes,      # the trimmed vector from step 1.5 (or 1.3 if you skipped it), same o
  sa2_id_col     = "SA2_MAIN16",  # check names(sf::st_read(...)) for your ASGS edition
  queen         = TRUE
)
```

This reorders the shapefile to `sa2_codes`'s order for you and errors if any SA2 in `sa2_codes` is missing from the shapefile — fix that before continuing rather than silently dropping units.

3. Validate before you run anything expensive

```
qa <- validate_W_adj(W, adj, census_sa2 = census_sa2)
qa$report           # full check-by-check table
stopifnot(qa$pass)   # stop here if anything failed
```

Read every FAIL line before proceeding — in particular, “W and adj have identical SA2 order” failing means `run_adsa_pipeline()` will silently delineate the wrong geography (right matrices, mismatched labels).

4. Run the pipeline

4.0 First, sanity-check the pipeline itself on prepared test data

Before trusting `run_adsa_pipeline()` on your real W/adj from steps 1–3, it's worth confirming the pipeline (and your R environment/install) actually works, on a small dataset with a **known correct answer** — so a bad result later is about your data, not a broken setup.

`examples/` ships exactly this: `W_N14.csv/adj_N14.csv` are a synthetic 14-BGU OD/adjacency pair engineered to have two obvious commuting communities, and `true_sol_N14.csv` is the partition that generated them. **This is test data, not your data** — swap back to the real W, adj, `row_W`, `col_W` from steps 1–3 in 4.1 below once you've confirmed the pipeline recovers it:

```
W_test      <- as.matrix(read.csv("examples/W_N14.csv", check.names = FALSE))
adj_test    <- as.matrix(read.csv("examples/adj_N14.csv", check.names = FALSE))
storage.mode(adj_test) <- "integer"
true_sol_test <- read.csv("examples/true_sol_N14.csv")$true_cluster

row_W_test  <- rowSums(W_test)
col_W_test  <- colSums(W_test)

# suggest_thresholds() only works because we happen to know the true
# partition for this toy case -- for real data use sc_table/self-containment
# (step 1.6) and your own Pop_min/Pop_max instead, as in 4.1 below.
th <- suggest_thresholds(true_sol_test, W_test, row_W_test, col_W_test, margin = 0.10)

res_test <- run_adsa_pipeline(
  W = W_test, adj = adj_test, row_W = row_W_test, col_W = col_W_test,
  SC_min = th$SC_min, Pop_min = th$Pop_min, Pop_max = th$Pop_max,
  r = 3, L = 100, l = 20, TO_samples = 50,
  seed = 42, verbose = FALSE
)

partitions_match(true_sol_test, res_test$best_sol)$match
count_misallocated(res_test$best_sol, W_test, adj_test, row_W_test, col_W_test)
```

gives you:

```
> partitions_match(true_sol_test, res_test$best_sol)$match
[1] TRUE
> count_misallocated(res_test$best_sol, W_test, adj_test, row_W_test, col_W_test)
[1] 0
```

If you don't get an exact match (0 misallocated, TRUE), something's wrong with the install or environment, not your Census data — sort that out before moving on. `examples/generate_toy_examples.R` regenerates this and two larger toy cases (N = 100, 520) if you want a tougher sanity check.

4.1 Run it on your real data

```
row_W <- rowSums(W)
col_W <- colSums(W)

res <- run_adsa_pipeline(
  W = W, adj = adj, row_W = row_W, col_W = col_W,
  SC_min = 0.55,      # from sc_table above -- pick a threshold most SA2s can plausibly meet
  Pop_min = 5000,     # from sc_table in step 1.6 -- see "Getting a data-driven Pop_min" there
  Pop_max = 50000,    # maximum, or leave as Inf
  r = 3,
  L = 20, l = 10, T0_samples = 20, # small first: size the run before scaling up
  seed = 2026
)
```

Start small (L = 20 or so) to see how long a trial takes on your data before committing to the default L = 1000. For a final delineation, don't rely on one seed — run several and compare (see `run_adsa_pipeline_parallel()`; the spread across seeds tells you whether the search found one stable structure or several competing ones).

What you get back

- `res$init_sol` — SARA's feasible starting partition (before annealing).
- `res$best_sol` — the final partition **after** annealing and misallocation refinement. Same length and order as `sa2_codes`: `best_sol[i]` is the LMA label of `sa2_codes[i]`.
- `res$lma_metrics` — one row per LMA: `cluster`, `pop`, `scss`, `scds`, `sc`, `n_bgus`.
- `res$gci`, `res$base_fitness`, `res$n_clusters` — recomputed for `best_sol`.
- `res$best_fitness`, `res$history`, `res$elite`, `res$op_tracker` — describe the *pre-refinement* search (see `?run_adsa_pipeline` for why these don't quite match `best_sol`).

Re-check feasibility — stage 3 (refinement) doesn't enforce population bounds:

```
with(res$lma_metrics, all(pop >= 5000 & pop <= 50000))
```

5. Convert the result back into ABS correspondence format

ABS geography correspondence files (e.g. “SA2 (2016) to SA4 (2016) Correspondence”) are one row per lower-level region, with the higher-level region’s code and name, and `RATIO_FROM_TO` / `RATIO_TO_FROM` columns. `build_sa2_lma_correspondence()` produces the same layout for your delineation, treating each LMA as the “higher-level region”. This table (§5.2–5.4) is the authoritative output; §5.5 optionally also attaches the cluster IDs to the SA2 shapefile geometry itself, for mapping:

5.1 Reattach the BGUs you set aside in step 1.5

`res$best_sol` only covers the trimmed `sa2_codes` — the zero-flow SA2s in `empty_bgu` (step 1.5) never went into the pipeline, so they’re not in `res$best_sol` either. But a real ABS-style correspondence table should still list every SA2 in the study area, occupied or not — you just can’t place these by commuting flow, since they have none. The next best thing is geographic proximity: `reattach_isolated_bgus()` assigns each one to the LMA of its nearest neighbour — first by shared border, falling back to nearest centroid distance in the rare case none of its borders made it into the run either (see `dev/abs_adjacency_from_shapefile.R` for the full logic, or `?reattach_isolated_bgus` once sourced).

It’s safe to call even if `empty_bgu` is empty (returns nothing to add) — always run this, then use the `_full` variables from here on instead of `sa2_codes/res$best_sol/row_W`:

```
empty_labels <- reattach_isolated_bgus(
  empty_codes = empty_bgu,          # from step 1.5
  sol         = res$best_sol,
  sa2_codes   = sa2_codes,
  shapefile_path = "SA2_2016_AUST_GDA2020.shp", # the *full* shapefile again --
  sa2_id_col   = "SA2_MAIN16"        # not subset to sa2_codes this time
)

sa2_codes_full <- c(sa2_codes, empty_bgu)
sol_full       <- c(res$best_sol, empty_labels)
row_W_full     <- c(row_W, setNames(rep(0L, length(empty_bgu)), empty_bgu))
```

Each reattached SA2 ends up with `RATIO_TO_FROM` = 0 in the correspondence table below — correct, since it contributes no population/commuters to its LMA, it just needed *somewhere* to belong geographically.

5.2 Build the correspondence table

```
sa2_names <- setNames(my_sa2_lookup$SA2_NAME_2016, my_sa2_lookup$SA2_MAINCODE_2016)

corr <- build_sa2_lma_correspondence(
  sol         = sol_full,
  sa2_codes   = sa2_codes_full,
  sa2_names   = sa2_names[sa2_codes_full], # optional; omit if you don't have names
  row_W       = row_W_full,                # population proxy for RATIO_TO_FROM
  lma_code_prefix = "LMA",
  sa2_year    = "2016", lma_year = "2016"
)

head(corr)
#   SA2_MAINCODE_2016 SA2_NAME_2016 LMA_CODE_2016          LMA_NAME_2016 RATIO_FROM_TO RATIO_TO_FROM
```

```
# 1      301011001 Fortitude Valley      LMA001 Local Labour Market Area 1      1      0.34
# ...
```

- **RATIO_FROM_TO** is always 1 here: this is a strict partition, so 100% of each SA2 belongs to exactly one LMA (unlike some ABS correspondences between geographies that don't nest cleanly).
- **RATIO_TO_FROM** is this SA2's share of its LMA's total population (or of **row_W**, if you didn't supply an actual population vector) — pass a real ERP population vector via **population =** for a more accurate figure than the commuting-flow proxy.

If you supplied real SA2 population instead of the **row_W** proxy — note this needs a value for the reattached SA2s too, 0 is correct for the same reason as **row_W_full** above:

```
my_sa2_population_full <- c(my_sa2_population, setNames(rep(0, length(empty_bgu)), empty_bgu))

corr <- build_sa2_lma_correspondence(
  sol = sol_full, sa2_codes = sa2_codes_full, sa2_names = sa2_names[sa2_codes_full],
  population = my_sa2_population_full # named or positional, aligned to sa2_codes_full
)
```

5.3 (Optional) attach per-LMA quality metrics

To carry **pop/sc** etc. from **res\$lma_metrics** alongside each SA2's assignment (handy for a single reportable table):

```
corr_full <- attach_lma_metrics(corr, sol_full, res$lma_metrics)
```

5.4 Export

```
write.csv(corr_full, "SA2_to_LMA_2016_Correspondence.csv", row.names = FALSE)
```

That CSV is now in the same shape analysts and downstream tools expect from an ABS correspondence product — joinable to any other SA2-keyed dataset, and readable without any **adsalmd**-specific knowledge.

5.5 (Optional) attach cluster IDs to the shapefile, for mapping

The CSV above is a *table* — it has no geometry, so nothing about it opens as a map. If you want the LMA delineation visible in QGIS/ArcGIS (to eyeball the result, or hand to someone who will), **export_lma_shapefile()** joins **corr_full** onto the SA2 boundary shapefile by SA2 code and writes the result back out as a shapefile, with **LMA_CODE_2016** (and, if you did 5.3, the attached metrics) as ordinary attribute columns you can symbolise or dissolve by:

```
export_lma_shapefile(
  correspondence = corr_full,
  shapefile_path = "SA2_2016_AUST_GDA2020.shp", # the full shapefile again, as in 5.1
  out_path       = "SA2_to_LMA_2016.shp",
  sa2_code_col   = "SA2_MAINCODE_2016",         # matches build_sa2_lma_correspondence()'s sa2_year
  sa2_id_col     = "SA2_MAIN16"                 # matches step 2's sa2_id_col
)
```

Shapefile column names are silently truncated to 10 characters (a DBF limitation, not a bug) — e.g. LMA_NAME_2016 becomes LMA_NAME_. Run `names(sf::st_read("SA2_to_LMA_2016.shp"))` afterwards if you need to know what a long or `attach_lma_metrics()`-prefixed column actually got called. This is exactly the same limitation ABS's own shapefile products have — the CSV correspondence table from 5.4 remains the authoritative, unambiguous output; treat this shapefile as a mapping convenience, not a replacement.

6. Full worked example (one script)

```
source("dev/abs_census_to_od.R")
source("dev/abs_adjacency_from_shapefile.R")
source("dev/validate_lma_inputs.R")
source("dev/lma_result_to_abs_correspondence.R")
library(adsalmd)

census_raw      <- read.csv("my_census_extract.csv", colClasses = "character")  # your real DataLab extract
census_raw$AGEP <- as.integer(census_raw$AGEP)
dzn_sa2_lookup  <- read.csv("DZN_SA2_2016_AUST.csv", colClasses = "character") # ABS-published concordance table

census_filtered <- filter_census_employed(census_raw)
census_sa2      <- convert_powp_to_sa2(census_filtered, dzn_sa2_lookup)
sa2_codes       <- sort(unique(c(census_sa2$origin_SA2, census_sa2$destination_SA2)))

W <- build_od_matrix(census_sa2, sa2_codes)

# step 1.5 -- only removes anything if sa2_codes came from a full
# study-area list rather than purely from census_sa2 (see step 1.3)
row_W <- rowSums(W); col_W <- colSums(W)
empty_bgu <- sa2_codes[row_W == 0 & col_W == 0]
sa2_codes <- setdiff(sa2_codes, empty_bgu)
W <- W[sa2_codes, sa2_codes]

adj <- build_adjacency_from_shapefile("SA2_2016_AUST_GDA2020.shp", sa2_codes) # ABS ASGS boundary file

qa <- validate_W_adj(W, adj, census_sa2 = census_sa2)
stopifnot(qa$pass)

row_W <- rowSums(W); col_W <- colSums(W)

res <- run_adsa_pipeline(
  W = W, adj = adj, row_W = row_W, col_W = col_W,
  SC_min = 0.55, Pop_min = 5000, Pop_max = 50000, r = 3,
  L = 1000, seed = 2026
)

# step 5.1 -- reattach empty_bgu by geographic proximity; safe to call
# even when empty_bgu is empty (returns nothing to add)
empty_labels <- reattach_isolated_bgus(
  empty_bgu, res$best_sol, sa2_codes, "SA2_2016_AUST_GDA2020.shp"
)
```

```

sa2_codes_full <- c(sa2_codes, empty_bgu)
sol_full      <- c(res$best_sol, empty_labels)
row_W_full    <- c(row_W, setNames(rep(0L, length(empty_bgu)), empty_bgu))

corr <- build_sa2_lma_correspondence(sol_full, sa2_codes_full, row_W = row_W_full)
full <- attach_lma_metrics(corr, sol_full, res$lma_metrics)
write.csv(full, "SA2_to_LMA_2016_Correspondence.csv", row.names = FALSE)

# step 5.5 (optional) -- same delineation, joined onto SA2 geometry for mapping
export_lma_shapefile(full, "SA2_2016_AUST_GDA2020.shp", "SA2_to_LMA_2016.shp")

```

7. Troubleshooting

Symptom	Likely cause
<code>validate_W_adj()</code> fails “W and adj have identical SA2 order”	You changed or re-sorted <code>sa2_codes</code> between calling <code>build_od_matrix()</code> and <code>build_adjacency_from_shapefile()</code> . Use the exact same vector for both.
<code>build_adjacency_from_shapefile()</code> errors “sa2_codes not found in shapefile”	Wrong <code>sa2_id_col</code> for your ASGS edition (check <code>names(sf::st_read(path))</code>), or your <code>sa2_codes</code> includes SA2s outside the shapefile’s coverage (e.g. a different state/edition).
<code>convert_powp_to_sa2()</code> drops most rows	Wrong <code>dzn_lookup_col/sa2_lookup_col</code> , or a DZN/SA2 vintage mismatch between the Census extract and the lookup file.
All SC values in <code>compute_self_containment()</code> are low	Your SA2s might be too small relative to real commuting patterns for the chosen study area — expect to need SARA/annealing to aggregate several SA2s per LMA; don’t set <code>SC_min</code> above what’s achievable pre-aggregation.
<code>lma_metrics</code> shows populations outside <code>Pop_min/Pop_max</code> after <code>run_adsa_pipeline()</code>	Expected — stage 3 (refinement) doesn’t enforce population bounds (see <code>?run_adsa_pipeline</code> , “Details”). Re-check and treat out-of-bounds LMAs as needing another pass or manual review.
Correspondence <code>RATIO_TO_FROM</code> looks wrong/uniform	You didn’t pass <code>population</code> or <code>row_W</code> , so every SA2 was weighted equally within its LMA (a warning is printed when this happens).
Some SA2s missing from the final correspondence table	You built <code>sa2_codes</code> from a full study-area list (step 1.3) and set some aside in step 1.5 for having no residents <i>and</i> no jobs recorded. Reattach them via step 5.1 before exporting — <code>res\$best_sol/sa2_codes</code> alone will never cover them.
Column missing/renamed after <code>export_lma_shapefile()</code>	Expected — the shapefile format truncates column names to 10 characters. Check <code>names(sf::st_read(out_path))</code> for what it actually got called, or use the CSV from step 5.4 (untruncated) as the authoritative output.

8. Where each piece lives

Step	Function	File
Filter microdata	<code>filter_census_employed()</code>	<code>dev/abs_census_to_od.R</code>
DZN $\rightarrow$ SA2	<code>convert_powp_to_sa2()</code>	<code>dev/abs_census_to_od.R</code>
Build <code>W</code>	<code>build_od_matrix()</code>	<code>dev/abs_census_to_od.R</code>
Remove zero-flow BGUs	inline code, no helper function	this guide, §1.5
Self-containment diagnostics	<code>compute_self_containment()</code>	<code>dev/abs_census_to_od.R</code>
Build <code>adj</code>	<code>build_adjacency_from_shapefile()</code>	<code>dev/abs_adjacency_from_shapefile.R</code>
QA <code>W/adj</code>	<code>validate_W_adj()</code>	<code>dev/validate_lma_inputs.R</code>
Run the pipeline	<code>run_adsa_pipeline()</code> / <code>run_adsa_pipeline_parallel()</code>	public package (<code>R/11_pipeline.R</code>)
Reattach zero-flow BGUs by proximity	<code>reattach_isolated_bgus()</code>	<code>dev/abs_adjacency_from_shapefile.R</code>
Result $\rightarrow$ correspondence table	<code>build_sa2_lma_correspondence()</code>	<code>dev/lma_result_to_abs_correspondence.R</code>
Attach LMA metrics	<code>attach_lma_metrics()</code>	<code>dev/lma_result_to_abs_correspondence.R</code>
Attach cluster IDs to the shapefile, for mapping	<code>export_lma_shapefile()</code>	<code>dev/lma_result_to_abs_correspondence.R</code>